INTERNAL · COPADO BUILDER POC

How We Built Copado Builder

Process overview for managers and stakeholders

AUDIENCE	LENGTH
Manager / stakeholders	About 2 pages
PURPOSE	**DATE**
Design approach, metadata, packaging & deploy options	July 23, 2026

1. What we built

Copado Builder is a Lightning app that turns a natural-language request into a Copado User Story, generates Salesforce metadata with Copado AI, then **Deploys** that package into a Dev org and **Commits** it to the story's feature branch — with guardrails (no Production, confirmation before risky steps, audited actions).

> **Chat → Improve / Create Story → Build → Deploy → Commit → (optional) Validate**

Work happens in a local Salesforce DX (SFDX) project, with Cursor as the coding assistant, then deploys into the Copado PSA org for live testing against Copado CI/CD and Copado AI.

2. Initial prompts and build approach

We did **not** start from a blank org with hand-clicked Setup. We started from a written product brief, then used Cursor to scaffold and iterate.

Kickoff prompt (condensed)

The first Cursor prompt asked for a Salesforce DX project that would:

1. Create custom objects for **Session**, **Message**, and **Action** (chat history + audit trail).
2. Build an LWC chat UI (`copadoBuilderChat`) with a status panel and action buttons.
3. Build Apex (`CopadoBuilderController`, `CopadoAiApiService`) so **all** Copado AI calls stay server-side (no API keys in the browser).
4. Use a Named Credential for Copado AI, with **mock mode** first so UI and scoring could ship before every live endpoint was wired.

5. Enforce guardrails: never show/deploy Production; require confirmation; require a User Story before build/deploy; log actions.

The same brief defined the UX ("What do you want to build?", story score, Improve / Create / Build / Deploy) and the phased plan: **mock story + scoring first**, then real Copado AI and Copado job templates.

How we worked after that

PHASE	WHAT WE DID
Scaffold	SFDX project under `force-app/`, objects, LWC, Apex, app/tab
Integrate	Named Credential + Copado AI dialogues; create/update User Stories
DevOps	Job Templates / Function to deploy AI packages into Dev, then commit
Iterate	Incremental "updates" zips (schema then code) via Workbench while testing

Later prompts were incremental (split Deploy vs Commit, Improve Story writes back to the linked US, fix commit selections for MDAPI paths, etc.) — always against the same SFDX source of truth.

3. Project structure and metadata types

Source layout (SFDX)

```
copado-builder/
  force-app/main/default/   ← editable source (objects, Apex, LWC, …)
  scripts/                  ← zip builders (MDAPI convert + package)
  copado-functions/         ← Function + Job Template docs (not in Salesforce zip)
  docs/                     ← process / install notes
```

Salesforce accepts **source format** (SFDX) or **Metadata API (MDAPI)** zips. We develop in SFDX; for Workbench we convert to MDAPI and zip.

Metadata we use — and why

TYPE	EXAMPLES	WHY
Custom Objects	`AI_Builder_Session__c`, Message, Action	Persist chats, scores, package JSON, job ids; audit every action
Custom Metadata	`Copado_Builder_Settings__mdt`	Org config (mock vs live, job template names) without hardcoding
User prefs object	`Copado_Builder_User_Preference__c`	Remember preferred project / Dev environment per user
Apex	Controller, AiApiService, DevopsService (+ tests)	UI API, Copado AI HTTP, CreateExecution / User Story metadata
LWC	`copadoBuilderChat`	Chat + status + Build / Deploy / Commit buttons
Aura + Quick Action	Global Action wrapper	Open Builder from the Salesforce `+` menu
App / Tab / FlexiPage	Copado Builder app	First-class app entry
Permission Set	`Copado_Builder_User`	Access without modifying profiles
Credentials + Remote Site	Named / External Credential	Secure outbound auth; secrets stay in Setup, not in git

Outside the Salesforce zip: Copado Function `builder_deploy_package` and Job Templates (`builder_deploy_only`, `builder_deploy_then_commit`, OOTB `sfdx_commit_1`). Those live in Copado Functions / Job Engine — documented under `copado-functions/`.

The app is a normal Salesforce DX package; Copado-native deploy/commit stays on Copado's job engine (same path UI Commit Changes uses).

4. How we zip and push today (Workbench)

Build the zips (local)

```
cd ~/Projects/copado-builder
./scripts/build-updates-zips.sh
```

That script:

1. Runs `sf project convert source` (SFDX → MDAPI).
2. Assembles smaller packages:
   - `copado-builder-updates-1-schema.zip` — object fields / CMD layout (first)
   - `copado-builder-updates-2-code.zip` — Apex, LWC, permission set, app pieces
3. Puts `package.xml` at the **zip root** (required for Metadata API / Workbench).

Why split schema vs code? New fields must land before Apex/LWC that reference them. Updates zips also **omit** redeploying live `Copado_Builder_Settings.Default` so mock/live toggles and org ids are not wiped.

Deploy with Workbench (current practice)

1. Open Workbench → log into the PSA org.
2. **migration** → **Deploy**.
3. Upload **schema zip first**, then **code zip**.
4. Check “Single Package”; run deploy.
5. Hard-refresh the Copado Builder tab.
6. Manual once (or after credential redeploy): paste Copado AI API key into the External Credential; set Job Template names on **Copado Builder Settings** → **Default**.

Loop for the POC: change source → rebuild zip → Workbench → test in org.

5. Other ways to push the same metadata

METHOD	HOW	WHEN IT FITS
Salesforce CLI	`sf project deploy start --source-dir force-app`	Day-to-day for developers; no zip needed
GitHub Actions	On push/PR: checkout → org login → deploy → optional tests	CI/CD for the Builder app; reviewable PRs
Copado pipeline	Promote this repo / MDAPI via User Story	Same governance customers use; shared environments
Change Sets	Setup upload	Poor fit for large LWC/Apex — avoid here
Unlocked / 2GP package	Version & install packages	Longer-term productization

Recommendation: Workbench for hotfixes; **GitHub Actions + Salesforce CLI** for repeatable sandbox deploys; **Copado** when promoting Builder through a shared pipeline. Functions / Job Templates still need documented one-time setup in Copado even if Salesforce metadata is automated.

6. What cannot go in the zip

- API keys / secrets (External Credential authentication parameters)
- Live CMD values we protect (Use Mock API, Organization Id, workspace id)
- Copado Function script and Job Template steps (Copado UI; see `copado-functions/`)
- Publisher Layout assignment for the Global Action

7. Summary for leadership

TOPIC	ANSWER
How it started	Product brief + Cursor prompt → SFDX POC with mock AI, then live Copado AI + Job Templates
Where code lives	Local git repo (`force-app`); not developed only in the org
What we deploy	Salesforce metadata (objects, Apex, LWC, credential shells, perms, app) + separate Copado Function/templates
How we push today	Scripted MDAPI zips → Workbench (schema, then code)
How we can push later	Salesforce CLI, GitHub Actions, or Copado promotions of the same source

---

Related runbooks: `DEPLOY.md`, `DEMO.md`, `copado-functions/PREFLIGHT.md`.
Document prepared for internal use — Copado Builder POC process overview.